Projeto 3 - Métodos Básicos Integro-Diferenciais

Enzo Reitano Capuano - 11370237

Prof. **José Abel Hoyos Neto** - *Introdução à Física Computacional (2026/2)*

Tarefa-1: Derivação Numérica

- Descrição e Lógica
- Código e Prints

Tarefa-2: Quadratura Numérica

- Descrição e Lógica
- Código e Prints

Tarefa-1: Derivação Numérica

Descrição e Lógica

Nesta tarefa utiliza-se aproximações numéricas para a primeira e a segunda derivadas totais de uma função de uma variável.

As equações utilizadas para aproximar a primeira derivada foram:

$$f'_{2f}(x) = \frac{f(x+h) - f(x)}{h} + \mathcal{O}(h); \tag{1}$$

$$f'_{2t}(x) = \frac{f(x) - f(x-h)}{h} + \mathcal{O}(h); \tag{2}$$

$$f'_{3s}(x) = \frac{f(x+h) - f(x-h)}{2h} + \mathcal{O}(h^2); \tag{3}$$

$$f'_{5s} = \frac{f(x-2h) - f(x+2h) + 8(f(x+h) - f(x-h))}{12h} + \mathcal{O}(h^4). \tag{4}$$

Para a segunda derivada foram usadas:

$$f''_{3s}(x) = \frac{f(x+h) + f(x-h) - 2f(x)}{h^2} + \mathcal{O}(h^2); \tag{5}$$

$$f''_{5s}(x) = \frac{16(f(x-h) + f(x+h)) - (f(x-2h) + f(x+2h) + 30f(x))}{12h^2} + \mathcal{O}(h^4); \tag{6}$$

A função trabalhada nesta tarefa foi

$$g(x) = e^{x^2}\,\mathrm{cotg}(2x), \tag{7}$$

sendo sua primeira e segunda derivadas, obtidas de forma analítica, iguais à

$$g'(x) = 2e^{x^2}[x\,\mathrm{cotg}(2x) - \mathrm{cosec}^2(2x)], \tag{8}$$

$$g''(x) = 2e^{x^2}[(1+2x^2)\,\mathrm{cotg}(2x) - 4x\,\mathrm{cosec}^2(2x) + 4\,\mathrm{cotg}(2x)\,\mathrm{cosec}^2(2x)]. \tag{9}$$

O programa começa definindo a função e calculando o resultado de suas derivadas obtidas analiticamente, além de definir o ponto no qual iremos calcular as derivadas, $x = 0, 1$.

Em seguida, começa-se um laço $i = 1, 12$, onde $h = 5^{-i}$. Neste laço, calcula-se e escreve-se as equações (1) à (6) para o ponto definido anteriormente em um arquivo output. Além disso, calcula-se os erros absolutos em comparação aos resultados obtidos analitcamente. Compara-se os erros da iteração atual com os erros da iteração anterior. Caso um erro da iteração atual seja *menor* do que a anterior, armazena-se o índice da iteração em um vetor.

Note, o índice armazenado corresponde àquele de *menor* erro para a *equação aproximativa* em questão (eqs. 1 a 6), portanto equivale à potência do *melhor* h respectivamente.

Por fim, o programa escreve no terminal os índices armazenados.

Código e Prints

```
      implicit real*8 (a-h,o-z)
      dimension e(6),e0(6),et(6)
      f(x) = exp(x*x)*cos(x+x)/sin(x+x)
      ft(x) = 2*exp(x*x)*(x/tan(x+x)-1/(sin(x+x)**2)) !1derivada ana
litica
      dt(x)=((1+2*x**2)/tan(2*x)-4*x/sin(2*x)**2)
      fft(x)= 2*exp(x*x)*(4/(sin(2*x)**2 *tan(2*x))+dt(x)) !2derivad
a analitica
```

```
      t=0.1d0
      open(10,file='tarefa1.dat')

      do i=1,6 !preenche o vetor 'erro anterior'
        e0(i)=1
      enddo

      do i=1,12
        h=5d0**(-i)
        f2f = (f(t+h)-f(t))/h
        f2t = (f(t)-f(t-h))/h
        f3s = (f2f+f2t)/2
        f5s = (f(t-2*h)+8*(f(t+h)-f(t-h))-f(t+2*h))/(12*h)
        ff3s=(f(t+h)+f(t-h)-2*(f(t)))/(h*h)
        ff5s=(16*(f(t+h)+f(t-h))-(f(t+2*h)+f(t-2*h)+30*f(t)))/(12*h*
h)
        write(10,*) h, f2f, f2t, f3s, f5s, ff3s, ff5s
        e(1)=abs(f2f-ft(t)) !vetor 'erro atual'
        e(2)=abs(f2t-ft(t))
        e(3)=abs(f3s-ft(t))
        e(4)=abs(f5s-ft(t))
        e(5)=abs(ff3s-fft(t))
        e(6)=abs(ff5s-fft(t))
        do k=1,6
            if(e(k).lt.e0(k)) then
                et(k)=i !acumula o índice de menor erro
            endif
            e0(k)=e(k)
        enddo
      enddo

      write(10,*) ft(t), fft(t)

      do i=1,6
        write(*,*) et(i)
      enddo

      close(10)
```

```
      end
```

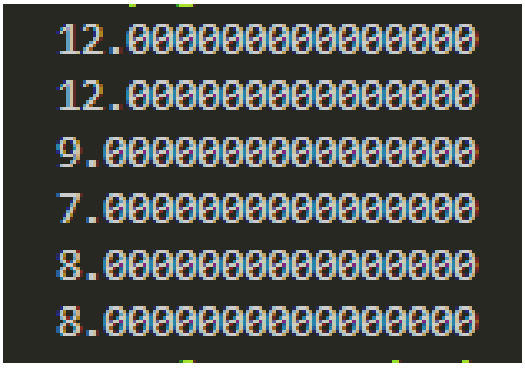


Figura 1: Melhor h para cada eq. usada



Mantendo a ordem das equações aproximativas utilizadas, a imagem acima indica respectivamente a potência de h que minimiza o erro absoluto entre o resultado numérico e o analítico. Nota-se que f'_{5s}, que tem erro $\mathcal{O}(h^4)$, atinge o melhor resultado para um h 5^5 vezes maior que f'_{2f}, que tem erro $\mathcal{O}(h)$, por exemplo, como esperado. Esse resultado também mostra que há um balanço entre diminuir o intervalo aproximativo usado (melhorando o erro da aproximação) mas usar números muito pequenos (aumentando o erro de ponto flutuante da precisão adotada), concluindo que o h ideal depende do caso individual analisado, não sendo fixo.

Tarefa-2: Quadratura Numérica

Descrição e Lógica

Nessa tarefa, utiliza-se três diferentes aproximações numéricas para integrais de funções de uma variável.

Utilizaremos a Regra do Trapézio, Simpson e Boole.

Código e Prints

```
      implicit real*8 (a-h,o-z)
      dimension e(3),e0(3),et(3)
```

```
      f(x)=exp(-x)*cos(x)

      pi=4*atan(1d0)
      ti=(1-exp(-2*pi))/2

      open(10,file='tarefa2.dat')

      do k=1,3
        e0(k)=1
      enddo

      do i=2,13
        n=2**i
        h=2*pi/n
        st=0
        ss=0
        sb=0

        do j=0,n
            xj=j*h
            fj=f(xj)

            if(j.eq.0.or.j.eq.n) then
                st=st+fj
                ss=ss+fj
                sb=sb+7*fj
            else
                st=st+2*fj

                if(mod(j,2).eq.1) then
                    ss=ss+4*fj
                else
                    ss=ss+2*fj
                endif

                if(mod(j,4).eq.1.or.mod(j,4).eq.3) then
                    sb=sb+32*fj
                else if(mod(j,4).eq.2) then
                    sb=sb+12*fj
                else
```

```
                    sb=sb+14*fj
                endif
            endif

          enddo

          st=h*st/2
          ss=h*ss/3
          sb=2*h*sb/45

          write(10,*) n,st,ss,sb

          e(1)=abs(st-ti)
          e(2)=abs(ss-ti)
          e(3)=abs(sb-ti)

          do k=1,3
              if(e(k).lt.e0(k)) then
                  et(k)=i
              endif
              e0(k)=e(k)
          enddo

      enddo

      write(10,*) ti

      do k=1,3
        write(*,*) et(k)
      enddo

      close(10)

      end
```

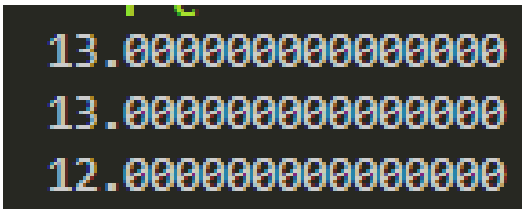


Figura 2: Melhor h para cada eq. usada